# Module 1 — Core Java for Spring Interviews

> For engineers coming from Node.js / MERN. Java is statically typed, compiled to bytecode, and runs on a managed runtime (the JVM) with real OS threads and a sophisticated garbage collector. Almost every Spring internal — bean proxies, `@Transactional`, thread pools, `CompletableFuture` — is built on the primitives in this module. Interviewers probe here to see whether you understand the *platform*, not just the syntax.

---

## 1.1 JVM Architecture, JDK vs JRE vs JVM

### Why Interviewers Ask This

They want to know if you understand what actually runs your code, how "write once run anywhere" works, and where to look when a service OOMs or stalls in production. Coming from Node (single V8 process, event loop), you must show you grasp the multi-threaded, managed-memory JVM model.

### Core Concept

- **JVM (Java Virtual Machine):** the abstract spec + a concrete process that loads bytecode (`.class`), verifies it, JIT-compiles hot paths to native code, manages memory, and runs threads. It is *platform-specific* (different binary per OS/arch) but runs *platform-independent* bytecode.
- **JRE (Java Runtime Environment):** JVM + core class libraries (`java.*`, `java.util.*`) needed to *run* apps. No compiler.
- **JDK (Java Development Kit):** JRE + developer tools (`javac`, `jar`, `javap`, `jstack`, `jmap`, `jcmd`, `jshell`). Needed to *build* apps.

Relationship: `JDK ⊃ JRE ⊃ JVM`.

### Internal Working

Source `.java` → `javac` → bytecode `.class` → class loader loads into JVM → bytecode verifier checks safety → interpreter executes; the **JIT compiler** (C1 client + C2 server, tiered compilation) profiles and compiles "hot" methods to optimized native code. HotSpot also does inlining, escape analysis (stack allocation of non-escaping objects), and deoptimization when assumptions break.

### Lifecycle / Execution Flow

```
.java --javac--> .class (bytecode)
                    |
                ClassLoader (load)
                    |
              Bytecode Verifier
                    |
        Interpreter  <----->  JIT (C1/C2)  --> native code cache
                    |
           Runtime Data Areas (Heap, Stacks, Metaspace, PC, Native)
                    |
             Execution Engine + GC threads
```

### ASCII Diagram — JVM Runtime Structure

```
+--------------------------- JVM PROCESS ---------------------------+
|  Class Loader Subsystem: Bootstrap -> Platform -> Application      |
|--------------------------------------------------------------------|
|  RUNTIME DATA AREAS                                                |
|   +---------------+   Shared across threads                       |
|   |     HEAP      |  (Young: Eden+S0/S1, Old)                     |
|   +---------------+                                               |
|   |   METASPACE   |  (class metadata, native memory)              |
|   +---------------+                                               |
|   Per-thread:                                                     |
|   [JVM Stack][PC Register][Native Method Stack]  x N threads      |
|--------------------------------------------------------------------|
|  EXECUTION ENGINE: Interpreter | JIT (C1/C2) | GC | Threads       |
+--------------------------------------------------------------------+
```

### Real Production Example

A Spring Boot service is packaged as a fat JAR and shipped in a container built `FROM eclipse-temurin:21-jre` (JRE only — smaller image, no compiler needed at runtime). You pass `-XX:MaxRAMPercentage=75` so the JVM sizes its heap relative to the container memory limit rather than the host.

### Advantages

Portability, mature JIT (often near-native throughput), rich tooling, managed memory (no manual free), massive ecosystem.

### Trade-offs

JVM warm-up latency (JIT needs profiling before peak performance), higher memory baseline than Node, GC pauses to reason about. Mitigations: AOT/CDS, GraalVM native image, `-XX:+UseAppCDS`.

### Common Mistakes

Shipping a full JDK in prod images; ignoring container memory limits (pre-JDK 10 JVMs read host memory, not cgroup limits → OOMKilled); confusing JRE and JDK.

### Performance Considerations

Tiered compilation reaches peak throughput after warm-up. Use `-Xss` for stack size, `-Xmx/-Xms` (or `MaxRAMPercentage`) for heap. CDS (Class Data Sharing) speeds startup.

### Debugging Techniques

`java -XX:+PrintFlagsFinal -version`, `jcmd <pid> VM.flags`, `jinfo`, `jstack` for thread dumps, `jmap`/`jcmd GC.heap_dump` for heap dumps.

### Common Interview Questions

- Difference between JDK, JRE, JVM?
- Is Java compiled or interpreted? *(Both: compiled to bytecode, then interpreted + JIT-compiled.)*
- What is the JIT and tiered compilation?

### Follow-up Questions

- How does the JVM decide a method is “hot”? *(invocation + back-edge counters.)*
- What is deoptimization? What is escape analysis?

### Hands-on Coding Exercise

Compile a class with `javac`, then run `javap -c` to inspect its bytecode and identify the `invokevirtual` / `invokedynamic` opcodes.

### Best Practices

Use a JRE (or `jlink`-trimmed runtime) in prod images; always set memory flags relative to the container; standardize on an LTS (17 or 21).

---

## 1.2 Java Memory Model: Heap, Stack, Metaspace

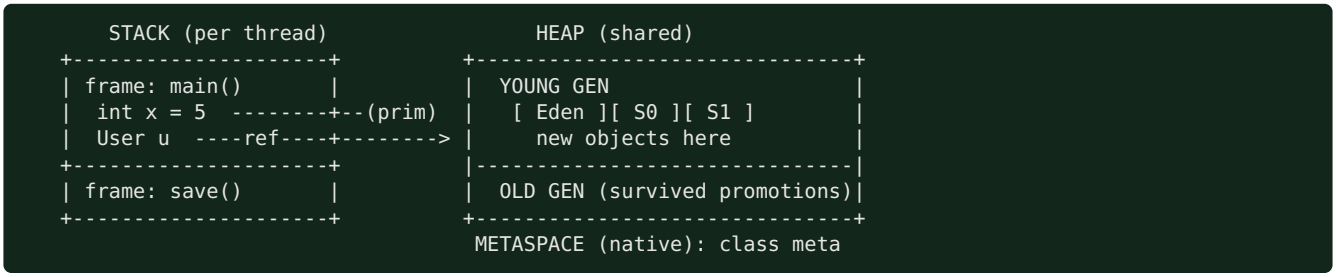
### Why Interviewers Ask This

Memory questions reveal whether you can diagnose OOMs, leaks, and GC pressure — the #1 production Java incident.

### Core Concept

- **Heap** — shared, GC-managed; all objects and arrays live here. Split into **Young Gen** (Eden + two Survivor spaces S0/S1) and **Old/Tenured Gen**.
- **Stack** — per-thread; stores frames (local variables, operand stack, return address). Primitives and *references* live here; the objects they point to live on the heap. `StackOverflowError` when too deep.
- **Metaspace** — native (off-heap) memory holding class metadata. Replaced PermGen in Java 8. Grows dynamically; cap with `-XX:MaxMetaspaceSize`.
- **PC register** and **native method stack** — per thread.
- **The JMM** (JSR-133) also defines *visibility/ordering* rules: `happens-before`, `volatile`, `synchronized`, `final` semantics.

### Internal Working & ASCII Diagram




Objects are allocated in Eden. A **minor GC** copies live objects Eden→Survivor, aging them; after N survivals they are **promoted** to Old Gen. A **major/full GC** collects Old Gen (more expensive).

### Real Production Example

`java.lang.OutOfMemoryError: Java heap space` in a report exporter that loads a whole result set into a `List`. Fix: stream results (JPA `Stream`, pagination) so objects stay short-lived in Young Gen and die at minor GC.

### Trade-offs / Common Mistakes

- Confusing the reference (stack) with the object (heap).
- Thinking Metaspace is on the heap — it isn't; a classloader leak (e.g. repeated hot redeploys, dynamic proxies) causes `OutOfMemoryError: Metaspace`.
- Huge thread counts × large `-Xss` → native OOM.

### Performance / Debugging

`jmap -histo:live <pid>`, heap dump + Eclipse MAT to find the dominator tree and retained size; `-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/dumps`.

### Interview Q / Follow-ups

- Where do primitives vs objects live? *(local primitives/refs on stack; objects on heap.)*
- What replaced PermGen and why? *(Metaspace; auto-grows in native memory.)*

- Explain `happens-before`. What does `volatile` guarantee? *(visibility + ordering, not atomicity of compound ops.)*

### Hands-on Exercise

Write a program that grows an unbounded `static List<byte[]>` and observe `OutOfMemoryError: Java heap space`; capture and open the heap dump.

### Best Practices

Prefer short-lived objects; avoid static collections that never release; set `-XX:MaxMetaspaceSize`; always enable heap-dump-on-OOM in prod.

---

## 1.3 Garbage Collection: G1, ZGC, Shenandoah

### Why Interviewers Ask This

GC tuning separates senior engineers. They want to know you can choose a collector by latency vs throughput requirements and read GC logs.

### Core Concept

GC reclaims unreachable objects (reachability from **GC roots**: stack refs, statics, JNI). Modern collectors are **generational** and mostly concurrent.

| Collector | Best for | Pause behavior | Notes |
|---|---|---|---|
| **Serial** | tiny heaps, single core | stop-the-world | `-XX:+UseSerialGC` |
| **Parallel** | batch/throughput | STW, multi-thread | max throughput |
| **G1** (default 9+) | balanced, large heaps | ~soft pause target | region-based |
| **ZGC** | very large heaps, low latency | <1ms, concurrent | colored pointers |
| **Shenandoah** | low latency (Red Hat) | <10ms | concurrent compaction |

### Internal Working

- **G1** divides the heap into equal **regions**; collects regions with most garbage first (“garbage first”). Uses SATB (snapshot-at-the-beginning) marking and honors a pause target (`-XX:MaxGCPauseMillis=200`).
- **ZGC / Shenandoah** do concurrent marking *and* concurrent relocation using load/read barriers (ZGC uses colored pointers + load barrier), so pauses stay sub-millisecond regardless of heap size (multi-TB).

### ASCII Diagram — Generational Copying

```
Eden fills ---> Minor GC ---> live copied to Survivor ---> age++
  |                                                          |
  +----- dead objects reclaimed                    age >= threshold
                                                             |
                                                        promote to OLD
OLD fills ---> concurrent mark (G1/ZGC) ---> reclaim/compact
```

### Real Production Example

A trading API with a strict p99 latency SLA switched from G1 to **ZGC** (`-XX:+UseZGC`) on a 64 GB heap; GC pauses dropped from ~120 ms to <1 ms, eliminating latency spikes, at a small throughput cost.

### Advantages / Trade-offs

G1: good default, predictable. ZGC/Shenandoah: tiny pauses but slightly lower throughput and higher CPU/barrier overhead. Parallel: highest throughput but long STW pauses.

### Common Mistakes

Blindly setting `-XX:MaxGCPauseMillis` too low (G1 shrinks young gen → frequent GC); ignoring allocation rate (real cause of GC pressure); calling `System.gc()`.

### Performance / Debugging

Enable unified GC logs: `-Xlog:gc*:file=gc.log:tags,uptime,level`. Analyze with GCEasy/GCViewer. Watch allocation rate, promotion rate, pause times.

### Interview Q / Follow-ups

- Difference between minor, major, full GC?
- Why is G1 the default? When choose ZGC?
- Can GC cause a memory leak? *(Yes — logical leaks: reachable-but-unused objects, e.g. static caches, unclosed listeners.)*
- What are GC roots?

### Hands-on Exercise

Run the same allocation-heavy program with `-XX:+UseParallelGC`, `-XX:+UseG1GC`, and `-XX:+UseZGC`; compare pause times in the GC logs.

### Best Practices

Start with G1; move to ZGC only for latency-critical, large-heap services; tune allocation rate before collector flags; always keep GC logs in prod.

---

## 1.4 Class Loading & Reflection

### Core Concept

Class loaders load classes lazily, following the **parent-delegation model**: **Bootstrap** (core `java.*`) → **Platform/Extension** → **Application/System** (your classpath). A child delegates to its parent first, preventing core classes from being overridden.

### ASCII Diagram

```
Bootstrap (rt/core)  <-- delegate
     ^
Platform loader      <-- delegate
     ^
Application loader (classpath / your JAR)
     ^
Custom loaders (Spring Boot's LaunchedURLClassLoader, web app loaders)
```

Spring Boot fat JARs use a **nested-jar** classloader that reads `BOOT-INF/lib`.

### Reflection

Inspect/modify classes at runtime: `Class.forName`, `getDeclaredMethods`, `setAccessible(true)`. Spring uses reflection heavily for DI, to instantiate beans, inject fields, and read annotations.

### Trade-offs / Mistakes / Debugging

Reflection is slower and breaks encapsulation; on JDK 17+ deep reflection into JDK internals needs `--add-opens`. `ClassNotFoundException` (not on classpath at runtime) vs `NoClassDefFoundError` (present at compile, missing/failed init at runtime) vs `ClassCastException`. Classloader leaks cause Metaspace OOM.

### Interview Q / Follow-ups

- Explain parent delegation and why it exists.

- `ClassNotFoundException` vs `NoClassDefFoundError`?
- How does Spring create beans it never sees the concrete type of? *(reflection + proxies.)*

### Hands-on Exercise

Use reflection to instantiate a class by name and invoke a private method via `setAccessible(true)`.

### Best Practices

Prefer `MethodHandles`/`VarHandle` over raw reflection for hot paths; cache `Method`/`Field` lookups; avoid reflection in tight loops.

---

## 1.5 Generics, Collections, Comparable vs Comparator

### Generics

Compile-time type safety with **type erasure** (types removed at runtime → no `new T[]`, no `instanceof T`). Wildcards: `? extends T` (producer, read), `? super T` (consumer, write) — **PECS: Producer Extends, Consumer Super**.

### Collections Cheat Sheet

| Interface | Impl | Ordering | Notes / Big-O |
|---|---|---|---|
| List | `ArrayList` | insertion | O(1) get, O(n) mid-insert |
| List | `LinkedList` | insertion | O(1) add ends, O(n) get |
| Set | `HashSet` | none | O(1) add/contains |
| Set | `LinkedHashSet` | insertion | predictable iteration |
| Set | `TreeSet` | sorted | O(log n), needs Comparable/Comparator |
| Map | `HashMap` | none | O(1) avg; treeifies buckets ≥8 |
| Map | `LinkedHashMap` | insertion/access | LRU cache base |
| Map | `TreeMap` | sorted keys | O(log n) |
| Map | `ConcurrentHashMap` | none | lock-striped, thread-safe |
| Queue | `ArrayDeque` | FIFO/LIFO | fast stack/queue |
| Queue | `PriorityQueue` | heap order | O(log n) offer/poll |

**HashMap internals:** array of buckets; key `hashCode()` spread → index; bucket holds a linked list, converted to a **red-black tree** when ≥8 entries (and table ≥64) to bound worst case at O(log n). Load factor 0.75 triggers resize (doubling).

### Comparable vs Comparator

- `Comparable<T>` — natural ordering, `compareTo`, one per class (`implements`).
- `Comparator<T>` — external/custom ordering, multiple allowed; `comparing`, `thenComparing`, `reversed`.

### ASCII — HashMap Bucket

```
index = (n-1) & spread(hash(key))
bucket[5] -> (k1,v1) -> (k2,v2) -> ... (list)  --treeify(>=8)-->  RB-tree
```

### Interview Q / Follow-ups

- How does `HashMap` work? What happens on collision / resize?
- Why override `equals` and `hashCode` together?
- `HashMap` vs `ConcurrentHashMap` vs `Collections.synchronizedMap`?
- `Comparable` vs `Comparator`; how to sort by multiple fields?

### Hands-on Exercise

Sort a `List<Employee>` by department asc then salary desc using `Comparator.comparing(Employee::dept).thenComparing(Employee::salary, reverseOrder())`.

### Common Mistakes / Best Practices

Mutable keys in a `HashMap`; overriding `equals` without `hashCode`; using `Vector`/`Hashtable` (legacy). Prefer `ConcurrentHashMap` over synchronized maps; program to interfaces (`List`, `Map`).

---

## 1.6 Streams, Optional, Functional Interfaces & Lambdas

### Core Concept

- **Functional interface:** exactly one abstract method (`@FunctionalInterface`). Key ones: `Function<T,R>`, `Supplier<T>`, `Consumer<T>`, `Predicate<T>`, `BiFunction`, `Runnable`, `Callable`.
- **Lambda:** anonymous implementation of a functional interface; compiled via `invokedynamic` (not an anonymous class) → cheaper.
- **Streams:** declarative pipelines — a **source** → **intermediate** (lazy: `map`, `filter`, `sorted`) → **terminal** (eager: `collect`, `reduce`, `forEach`). Support `parallel()` (uses common ForkJoinPool).
- **Optional:** container to express "maybe absent" and avoid `NullPointerException`; use `map`, `filter`, `orElseGet`, `orElseThrow` — **never** call `get()` blindly.

### ASCII — Stream Pipeline

```
source -> filter -> map -> sorted -> collect
 (lazy intermediate ops build a pipeline; nothing runs)
                            └── terminal op triggers execution
```

### Real Production Example

```
Map<String, List<Order>> byUser = orders.stream()
    .filter(o -> o.getTotal().compareTo(BigDecimal.TEN) > 0)
    .collect(Collectors.groupingBy(Order::getUserId));
```

### Trade-offs / Common Mistakes

Streams are less debuggable than loops and can be slower for trivial cases; `parallelStream()` on small data or with shared mutable state hurts (and shares the common pool with the whole app). `Optional` as a *field* or method parameter is an anti-pattern — use it as a return type.

### Interview Q / Follow-ups

- `map` vs `flatMap`? Intermediate vs terminal ops? Why lazy?
- When is `parallelStream()` a bad idea?
- Why `orElseGet` over `orElse` for expensive defaults? *(`orElse` always evaluates its argument.)*

### Hands-on Exercise

Given `List<Transaction>`, compute total amount per currency and the top-3 highest transactions using streams.

### Best Practices

Keep pipelines side-effect-free; return `Optional` (not null) for "maybe"; avoid parallel streams unless data is large, CPU-bound, and stateless.

---

## 1.7 Exception Handling

### Core Concept

- **Checked** (`extends Exception`) — must be declared/handled; for recoverable conditions (`IOException`).
- **Unchecked** (`extends RuntimeException`) — programming errors (`NullPointerException`, `IllegalArgumentException`); not required to declare.
- **Error** (`OutOfMemoryError`) — don't catch.
- `try-with-resources` auto-closes `AutoCloseable`; `finally` for cleanup; multi-catch `catch (A | B e)`.

### Common Mistakes / Best Practices

Swallowing exceptions (`catch (Exception e) {}`); catching `Throwable`; losing the cause (always chain: `throw new ServiceException("msg", e)`); using exceptions for control flow. In Spring, translate to a global `@ControllerAdvice`.

### Interview Q

Checked vs unchecked; when to create custom exceptions; why is `throws Exception` bad on public APIs?

---

## 1.8 Concurrency: Threads, ExecutorService, Thread Pools, CompletableFuture

### Why Interviewers Ask This

This is the biggest gap for Node developers. Java uses real OS threads and shared mutable memory, so you must reason about visibility, atomicity, and pools.

### Core Concept

- **Thread** — unit of execution; `Runnable`/`Callable`. Creating threads per task is expensive → use pools.
- **ExecutorService / Thread Pool** — reuse a fixed set of worker threads pulling from a task queue. Create via `ThreadPoolExecutor` (avoid `Executors` factories with unbounded queues in prod).
- **Key params:** `corePoolSize`, `maxPoolSize`, `keepAlive`, `workQueue`, `RejectedExecutionHandler`.
- **CompletableFuture** — composable async: `supplyAsync`, `thenApply`, `thenCompose` (flatMap), `thenCombine`, `exceptionally`, `allOf`. This is the closest thing to Node's Promises, but backed by a thread pool.
- **Synchronization:** `synchronized`, `ReentrantLock`, `volatile`, atomics (`AtomicInteger`, `LongAdder`), `ConcurrentHashMap`.
- **Virtual threads (Java 21, Project Loom):** cheap user-mode threads for high I/O concurrency — millions per JVM.

### ASCII — Thread Pool

```
submit(task) -> [ work queue ]  ->  worker1  worker2 ... workerN
                      |                 (core..max threads)
                queue full & max reached -> RejectedExecutionHandler
```

### CompletableFuture flow

```
supplyAsync(fetchUser) --thenCompose--> fetchOrders(user)
       \--thenCombine(fetchProfile)--> merge --exceptionally--> fallback
```

### Real Production Example

Aggregating three downstream calls in parallel and merging with a timeout:

```
CompletableFuture<User>    u = supplyAsync(() -> userClient.get(id), pool);
CompletableFuture<Profile> p = supplyAsync(() -> profileClient.get(id), pool);
return u.thenCombine(p, ProfileView::of)
        .orTimeout(500, MILLISECONDS)
        .exceptionally(ex -> ProfileView.fallback(id));
```

### Trade-offs / Common Mistakes

- Using the default `ForkJoinPool.commonPool` for blocking I/O (starves it) — pass an explicit pool to `*Async`.
- Unbounded queues (`Executors.newFixedThreadPool`) hide backpressure → OOM.
- Data races from unsynchronized shared mutable state; deadlocks from lock ordering; `.get()` blocking defeats async.
- Thread pool sizing: CPU-bound ≈ #cores; I/O-bound larger (Little's Law).

### Performance / Debugging

Thread dump `jstack <pid>` (find BLOCKED/WAITING, deadlocks); look for pool saturation, `RejectedExecutionException`, threads stuck on locks. Async Profiler / JFR for CPU + lock contention.

### Interview Q / Follow-ups

- Runnable vs Callable? `Future` vs `CompletableFuture`?
- Explain `ThreadPoolExecutor` params and rejection policies.
- How do you size a thread pool? CPU-bound vs I/O-bound.
- `synchronized` vs `ReentrantLock`? What does `volatile` guarantee?
- What are virtual threads and when do they help? *(massive blocking I/O concurrency; not CPU-bound work.)*
- How to prevent/detect deadlock? *(consistent lock ordering, tryLock w/ timeout, thread dump.)*

### Hands-on Exercise

Build a `ThreadPoolExecutor(4, 8, 60s, ArrayBlockingQueue(100), CallerRunsPolicy)`, submit 1000 tasks, and observe backpressure via `CallerRunsPolicy`.

### Best Practices

Always use bounded queues + explicit pools; name threads (`ThreadFactory`) for debuggability; prefer `CompletableFuture`/structured concurrency over raw threads; never share mutable state without synchronization.

---

## Module 1 — One-Page Cheat Sheet

| Topic | Must-know |
|---|---|
| JDK/JRE/JVM | JDK⊃JRE⊃JVM; JIT tiered (C1/C2); bytecode is portable |
| Memory | Heap (Young Eden+S0/S1, Old), Stack (per-thread frames), Metaspace (native) |
| GC | G1 default; ZGC/Shenandoah = sub-ms pauses; roots = stack/statics/JNI |
| Collections | HashMap treeifies at 8; ConcurrentHashMap for concurrency; PECS |
| equals/hashCode | Override together; immutable keys |
| Streams | Lazy intermediate + eager terminal; avoid parallel by default |
| Optional | Return type only; `orElseGet` for expensive defaults |
| Exceptions | Checked=recoverable; chain cause; never swallow |
| Concurrency | Bounded pools + explicit executor; CompletableFuture composes; volatile≠atomic |
| Virtual threads | Java 21; great for blocking I/O, not CPU-bound |

## Module 1 — Top Interview Questions

1. Explain JVM memory areas and where objects vs references live.
2. How does `HashMap` work internally (collision, treeify, resize)?
3. Compare G1, ZGC, Shenandoah — when use each?
4. `Comparable` vs `Comparator`; sort by multiple fields.
5. `map` vs `flatMap`; why are streams lazy?
6. Explain `ThreadPoolExecutor` parameters and rejection policies.
7. `Future` vs `CompletableFuture`; compose parallel calls with timeout/fallback.
8. What does `volatile` guarantee; `synchronized` vs `ReentrantLock`.
9. Checked vs unchecked exceptions; exception chaining.
10. Virtual threads — what problem do they solve?

## Module 1 — Common Mistakes

- Overriding `equals` without `hashCode`.
- Blocking I/O on the common ForkJoinPool.
- Unbounded thread-pool queues → OOM.
- Calling `Optional.get()` without checking; `Optional` as a field.
- Swallowing exceptions / losing the root cause.

## Module 1 — Mock Interview (5 rapid-fire)

1. *"Our service is OOMing with `Java heap space`. Walk me through diagnosis."* → enable heap-dump-on-OOM, take dump, open in MAT, find dominators/retained size, look for unbounded static collections or a loaded-everything query; fix by streaming/paginating.
2. *"p99 latency spikes every few minutes."* → likely GC pauses; check GC logs, allocation rate; consider ZGC or reduce allocation.
3. *"Two threads hang forever."* → deadlock; take `jstack`, look for "Found one Java-level deadlock"; enforce lock ordering.
4. *"Why not just create a thread per request like Node handles requests?"* → OS threads are heavy (~1MB stack); use pools or virtual threads.
5. *"Combine three REST calls, return in 300ms or fallback."* → three `supplyAsync` on a bounded pool + `allOf` / `thenCombine` + `orTimeout` + `exceptionally`.

**Next** → Module 2: Spring Core (IoC, DI, bean lifecycle & the container).